

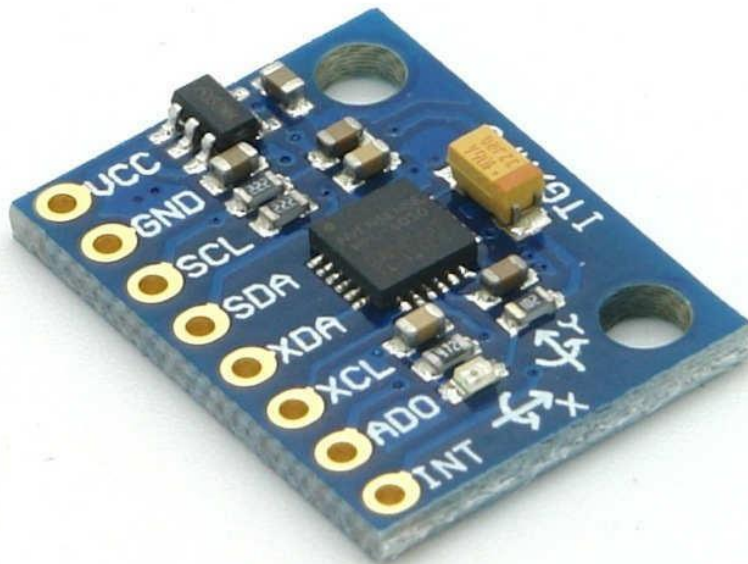
3- Axis Motion potion (Roll, Pitch, Yaw) for visualization

Aim:

To design and implement a gesture-based control system using the MPU6050 sensor that detects motion in **roll, pitch, and yaw axes**, and visually represents the movement in real time.

Components Required:

Sno	Component Name	Quantity
1.	ESP32C6 dev Module	1
2.	MPU6050 Sensor	1
3.	Breadboard	1
4.	Jumper Wires	As needed

Components in detail:

MPU6050 sensor module is complete 6-axis Motion Tracking Device. It combines 3-axis Gyroscope, 3-axis Accelerometer and Digital Motion Processor all in small package.

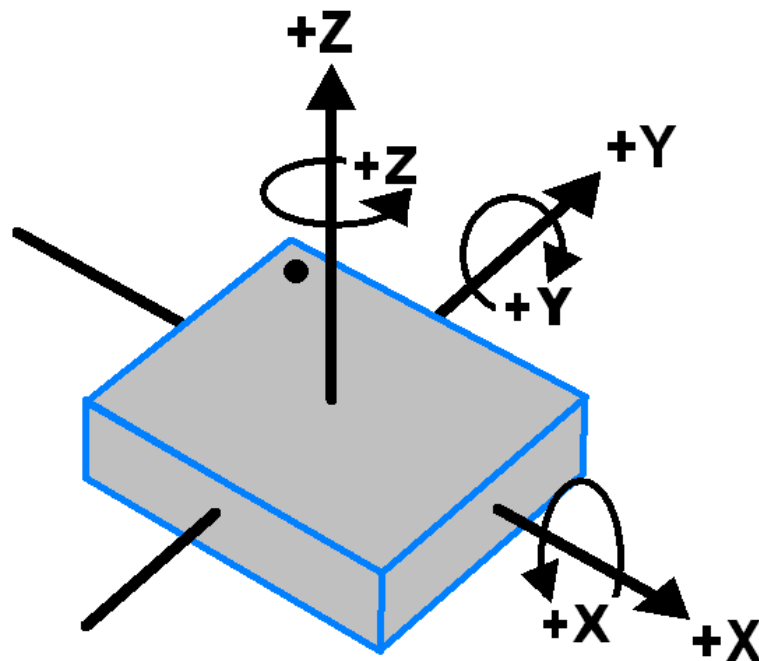
NOTE: Also, the sensor has an additional feature of on-chip Temperature sensor. It has I2C bus interface to communicate with the microcontrollers.

It has Auxiliary I2C bus to communicate with other sensor devices like 3-axis Magnetometer, Pressure sensor etc.

If the 3-axis Magnetometer is connected to the auxiliary I2C bus, then the MPU6050 can provide complete 9-axis Motion Fusion output.

3-Axis Gyroscope

The MPU6050 consists of a 3-axis Gyroscope with Micro Electromechanical System (MEMS) technology. It is used to detect rotational velocity along the X, Y, Z axes as shown in below figure.



MPU-6050 Orientation & Polarity of Rotation

When the gyros are rotated about any of the sense axes, the Coriolis Effect causes a vibration that is detected by a MEM inside MPU6050.

The resulting signal is amplified, demodulated, and filtered to produce a voltage that is proportional to the angular rate.

This voltage is digitized using 16-bit ADC to sample each axis.

The full-scale range of output are +/- 250, +/- 500, +/- 1000, +/- 2000.

It measures the angular velocity along each axis in degrees per second unit.

3-Axis Accelerometer

The MPU6050 consists of a 3-axis Accelerometer with Micro Electromechanical (MEMs) technology. It is used to detect angle of tilt or inclination along the X, Y and Z axes as shown in below figure.

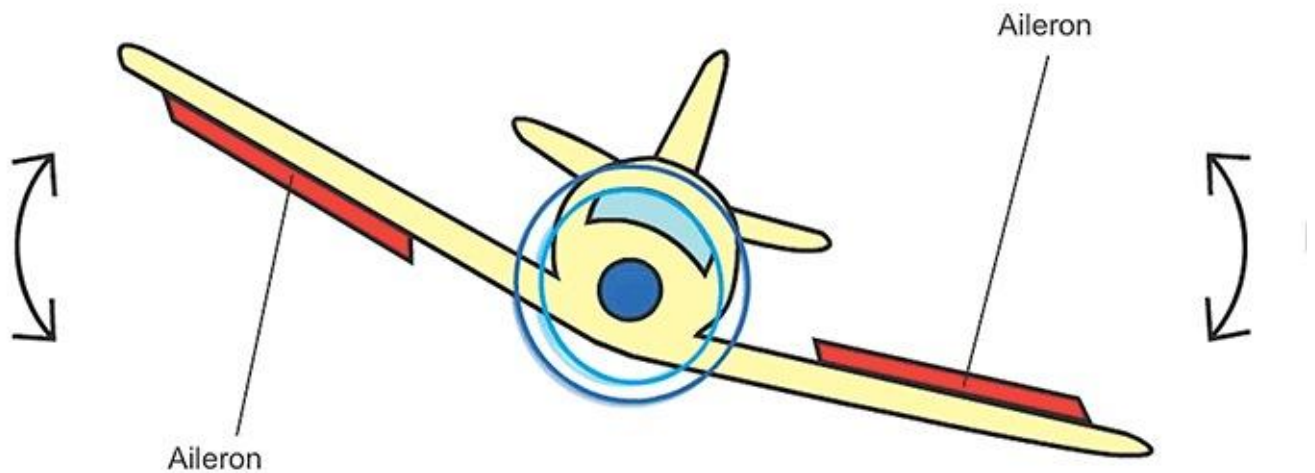


Fig: Roll

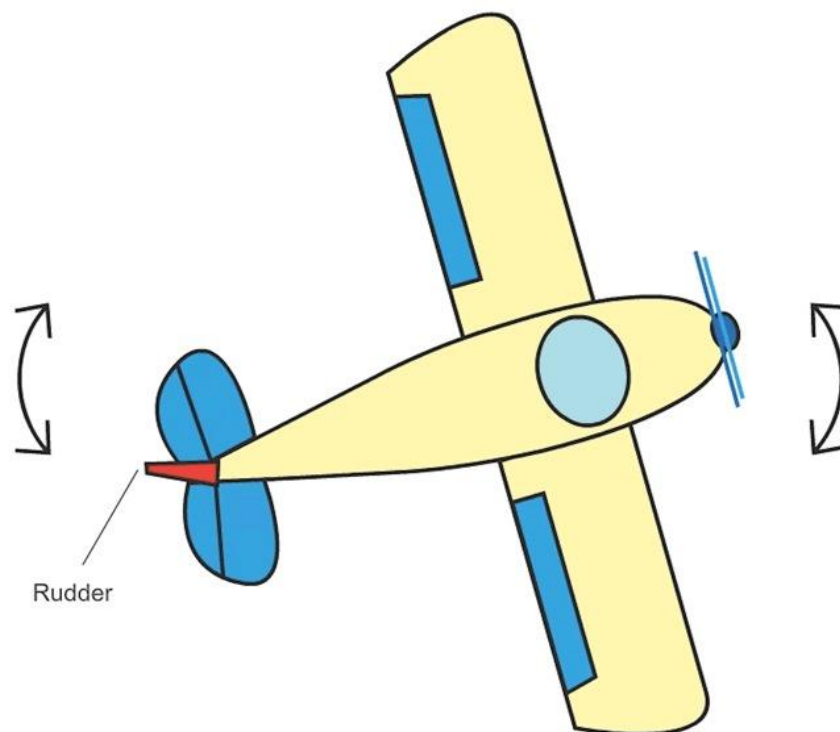


Fig: Yaw

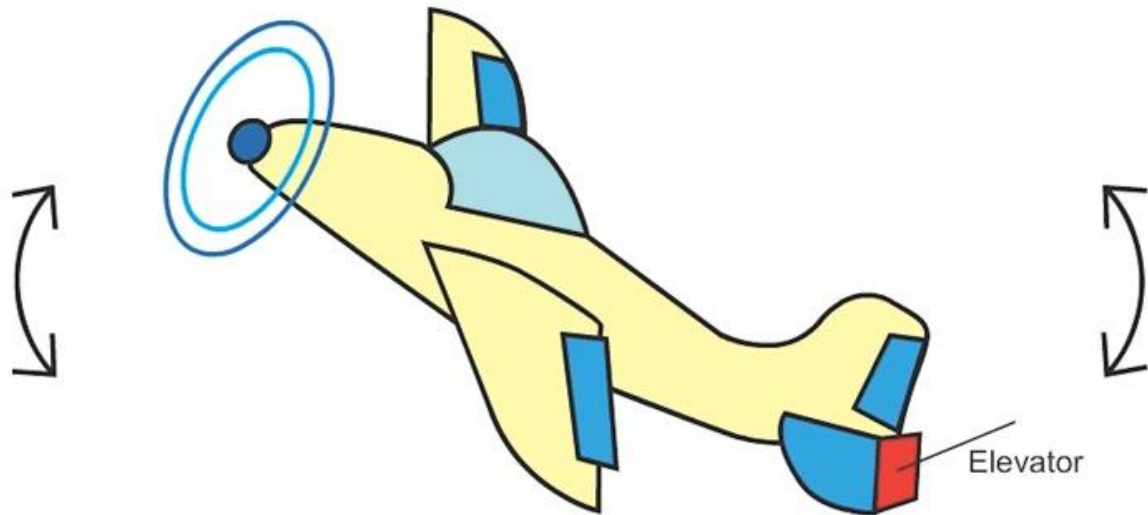


Fig: Pitch

What are Roll, Pitch, and Yaw?

Imagine an airplane with three invisible rods passing right through its centre one from nose to tail, one from wing to wing, and one straight up and down. Now think of the plane doing a little dance around each of these rods—tilting, nodding, and turning like it has a personality of its own. These fun movements are what we call roll, pitch, and yaw, and they help us understand how anything moves in 3D space!

- **Roll (X-axis):** Rotation around the front-to-back axis, causing the object to tilt left or right.
- **Pitch (Y-axis):** Rotation around the side-to-side axis, causing the object to move up or down (forward/backward tilt).
- **Yaw (Z-axis):** Rotation around the vertical axis, causing the object to turn left or right without tilting.

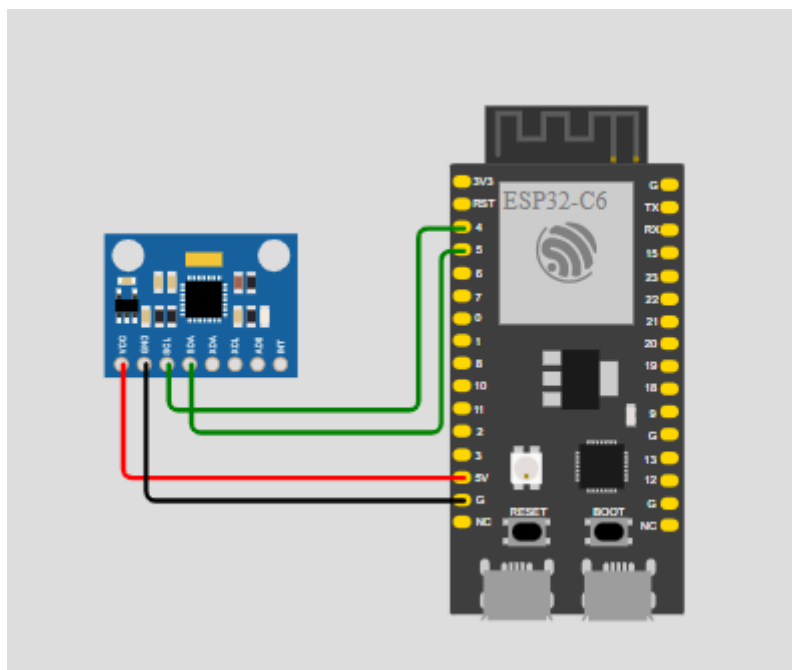
The MPU-6050 module has 8 pins as listed below:

1. **INT:** Interrupt digital output pin.
2. **AD0:** I2C Slave Address LSB pin. This is 0th bit in 7-bit slave address of device. If connected to VCC then it is read as logic one and slave address changes.
3. **XCL:** Auxiliary Serial Clock pin. This pin is used to connect other I2C interface enabled sensor SCL pin to MPU-6050.

4. **XDA**: Auxiliary Serial Data pin. This pin is used to connect other I2C interface enabled sensor SDA pin to MPU-6050.
5. **SCL**: Serial Clock pin. This pin to connected to microcontrollers SCL pin.
6. **SDA**: Serial Data pin. This pin is connected to microcontrollers SDA pin.
7. **GND**: Ground pin. This requires a ground connection.
8. **VCC**: Power supply pin. This requires a +5V DC supply.

Note: For detailed theory refer to this [Link](#)

Circuit Diagram:



Software - Implementing the gesture project

- Download the library called “MPU6050” by the author electronic cats in library Manager of Arduino IDE itself.
- The Project requires a software tool called “Processing”, Please find the [link](#) to download the tool.
- Please find the GitHub repository [link](#) for the code.

Note: While downloading please cross check the software version should be (4.4.7).

Implementing the code:

Note: The Below code to be uploaded in ESP32C6 Dev board using Arduino IDE to read the angle.

```
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

#define ALPHA    0.96
#define LOW_PASS 0.1
#define DEAD_ZONE 0.8
#define SEND_INTERVAL 30

float pitch = 0, roll = 0, yaw = 0;
float filtPitch = 0, filtRoll = 0, filtYaw = 0;
unsigned long lastTime = 0;
unsigned long lastSend = 0;

void setup() {
  Serial.begin(115200);
  Wire.begin(5, 4); // SDA = 5 and SCL = 4
  mpu.initialize();

  if (mpu.testConnection()) {
    Serial.println("MPU6050 connected!");
  } else {
    Serial.println("MPU6050 connection failed!");
  }

  mpu.setXGyroOffset(0);
  mpu.setYGyroOffset(0);
  mpu.setZGyroOffset(0);
  mpu.setXAccelOffset(0);
  mpu.setYAccelOffset(0);
  mpu.setZAccelOffset(0);

  lastTime = millis();
}
```

```

void loop() {
    unsigned long now = millis();
    float dt = (now - lastTime) / 1000.0;
    lastTime = now;

    int16_t ax, ay, az, gx, gy, gz;
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

    float gyroX = gx / 131.0;
    float gyroY = gy / 131.0;
    float gyroZ = gz / 131.0;

    float accelPitch = atan2(ay, az) * 180.0 / PI;
    float accelRoll = atan2(ax, az) * 180.0 / PI;

    pitch = ALPHA * (pitch + gyroX * dt) + (1.0 - ALPHA) * accelPitch;
    roll = ALPHA * (roll + gyroY * dt) + (1.0 - ALPHA) * accelRoll;
    yaw += gyroZ * dt;

    filtPitch = LOW_PASS * pitch + (1.0 - LOW_PASS) * filtPitch;
    filtRoll = LOW_PASS * roll + (1.0 - LOW_PASS) * filtRoll;
    filtYaw = LOW_PASS * yaw + (1.0 - LOW_PASS) * filtYaw;

    static float lastSentPitch = 0, lastSentRoll = 0, lastSentYaw = 0;
    bool changed = abs(filtPitch - lastSentPitch) > DEAD_ZONE ||
        abs(filtRoll - lastSentRoll) > DEAD_ZONE ||
        abs(filtYaw - lastSentYaw) > DEAD_ZONE;

    if ((now - lastSend >= SEND_INTERVAL) && changed) {
        Serial.print(filtPitch); Serial.print(",");
        Serial.print(filtRoll); Serial.print(",");
        Serial.println(filtYaw);

        lastSentPitch = filtPitch;
        lastSentRoll = filtRoll;
        lastSentYaw = filtYaw;
        lastSend = now;
    }
}

```


Note: The Below code to be uploaded in Processing Software, while uploading please **close the Serial Monitor in Arduino IDE**, then only the project works.

- While uploading please verify the COM port in your laptop and mention that COM port in the Below code.

```
import processing.serial.*;

Serial port;
float pitch = 0, roll = 0, yaw = 0;
float smoothPitch = 0, smoothRoll = 0, smoothYaw = 0;
float lerpSpeed = 0.15;

// Calibration offsets — captured when user presses C
float offsetPitch = 0;
float offsetRoll = 0;
float offsetYaw = 0;
boolean calibrated = false;

// Button
float btnX = 20, btnY = 590, btnW = 220, btnH = 38;
boolean btnHover = false;

void setup() {
  size(900, 650, P3D);
  port = new Serial(this, "COM3", 115200); // Change COM3 to your port
  port.bufferUntil('\n');
}

void draw() {
  background(15, 15, 30);

  float adjPitch = pitch - offsetPitch;
  float adjRoll = roll - offsetRoll;
  float adjYaw = yaw - offsetYaw;

  smoothPitch = lerp(smoothPitch, adjPitch, lerpSpeed);
  smoothRoll = lerp(smoothRoll, adjRoll, lerpSpeed);
  smoothYaw = lerp(smoothYaw, adjYaw, lerpSpeed);

  hint(DISABLE_DEPTH_TEST);
```

```

camera();
noLights();
drawHUD();
drawButton();
hint(ENABLE_DEPTH_TEST);

ambientLight(60, 60, 80);
directionalLight(255, 240, 220, -1, -1, -1);
directionalLight(40, 60, 100, 1, 1, 0.5);

translate(width / 2, height / 2 - 20, 0);
rotateX(radians(smoothPitch));
rotateY(radians(smoothYaw));
rotateZ(radians(-smoothRoll));
scale(2.2);

drawJet();
}

void calibrate() {
    offsetPitch = pitch;
    offsetRoll = roll;
    offsetYaw = yaw;
    smoothPitch = 0;
    smoothRoll = 0;
    smoothYaw = 0;
    calibrated = true;
}

void mousePressed() {
    if (mouseX > btnX && mouseX < btnX + btnW &&
        mouseY > btnY && mouseY < btnY + btnH) {
        calibrate();
    }
}

void mouseMoved() {
    btnHover = (mouseX > btnX && mouseX < btnX + btnW &&
        mouseY > btnY && mouseY < btnY + btnH);
}

void keyPressed() {
    if (key == 'c' || key == 'C') calibrate();
}

```

```

}

void drawButton() {
  if (calibrated && !btnHover) {
    fill(20, 80, 40);
  } else if (btnHover) {
    fill(50, 100, 200);
  } else {
    fill(30, 60, 130);
  }
  noStroke();
  rect(btnX, btnY, btnW, btnH, 7);

  stroke(180, 210, 255, 180);
  strokeWeight(1.5);
  float cx = btnX + 20, cy = btnY + btnH / 2;
  line(cx - 7, cy, cx + 7, cy);
  line(cx, cy - 7, cx, cy + 7);
  ellipse(cx, cy, 10, 10);
  noStroke();

  fill(200, 220, 255);
  textSize(13);
  textAlign(LEFT, CENTER);
  text(calibrated ? "Re-calibrate Horizon [C]" : "Calibrate Horizon [C]",
    btnX + 38, btnY + btnH / 2);
}

void drawHUD() {
  fill(0, 0, 0, 140);
  noStroke();
  rect(14, 14, 220, 130, 8);

  fill(180, 210, 255);
  textSize(12);
  textAlign(LEFT);
  text("MPU6050 | ESP32C6", 26, 34);

  textSize(11);
  fill(calibrated ? color(80, 200, 100) : color(255, 160, 60));
  text(calibrated ? "Calibrated" : "Place flat and press C", 26, 52);

  textSize(14);

```

```

fill(100, 220, 140);
text("PITCH " + nf(smoothPitch, 3, 1) + "°", 26, 72);
fill(100, 180, 255);
text("ROLL " + nf(smoothRoll, 3, 1) + "°", 26, 92);
fill(255, 200, 80);
text("YAW " + nf(smoothYaw, 3, 1) + "°", 26, 112);

drawBar(26, 126, smoothPitch, color(100, 220, 140));
drawBar(96, 126, smoothRoll, color(100, 180, 255));
drawBar(166, 126, smoothYaw, color(255, 200, 80));
}

void drawBar(float x, float y, float val, color c) {
  float w = 60;
  float filled = map(constrain(val, -90, 90), -90, 90, 0, w);
  fill(40, 40, 60);
  noStroke();
  rect(x, y, w, 7, 3);
  fill(c);
  rect(x, y, filled, 7, 3);
}

void drawJet() {
  noStroke();

  fill(220, 225, 235);
  pushMatrix();
  translate(-80, 0, 0);
  scale(2.5, 0.35, 0.35);
  sphere(30);
  popMatrix();

  fill(200, 205, 215);
  pushMatrix();
  translate(20, 0, 0);
  box(180, 22, 20);
  popMatrix();

  fill(185, 190, 200);
  pushMatrix();
  translate(118, 0, 0);
  scale(1.0, 0.55, 0.55);
  sphere(22);

```

```
popMatrix();
```

```
fill(170, 175, 195);  
pushMatrix();  
translate(30, 3, 0);  
rotateY(radians(18));  
rotateZ(radians(3));  
scale(1.0, 0.05, 1.0);  
box(150, 1, 70);  
popMatrix();
```

```
fill(160, 165, 185);  
pushMatrix();  
translate(-10, 3, -48);  
rotateY(radians(30));  
scale(0.5, 0.05, 0.4);  
box(100, 1, 40);  
popMatrix();
```

```
fill(170, 175, 195);  
pushMatrix();  
translate(30, 3, 0);  
rotateY(radians(-18));  
rotateZ(radians(-3));  
scale(1.0, 0.05, 1.0);  
box(150, 1, 70);  
popMatrix();
```

```
fill(160, 165, 185);  
pushMatrix();  
translate(-10, 3, 48);  
rotateY(radians(-30));  
scale(0.5, 0.05, 0.4);  
box(100, 1, 40);  
popMatrix();
```

```
fill(155, 160, 178);  
pushMatrix();  
translate(95, -22, 0);  
scale(0.08, 1.0, 1.0);  
box(1, 44, 38);  
popMatrix();
```

```
fill(150, 155, 172);  
pushMatrix();  
translate(90, 10, -10);  
rotateZ(radians(-40));  
scale(0.06, 1.0, 0.5);  
box(1, 22, 14);  
popMatrix();
```

```
pushMatrix();  
translate(90, 10, 10);  
rotateZ(radians(40));  
scale(0.06, 1.0, 0.5);  
box(1, 22, 14);  
popMatrix();
```

```
fill(120, 125, 140);  
pushMatrix();  
translate(105, 2, 0);  
scale(1.0, 0.45, 0.45);  
sphere(24);  
popMatrix();
```

```
fill(80, 82, 95);  
pushMatrix();  
translate(128, 2, 0);  
rotateY(radians(90));  
noFill();  
stroke(80, 82, 95);  
strokeWeight(5);  
ellipse(0, 0, 26, 22);  
noStroke();  
popMatrix();
```

```
fill(255, 140, 30, 200);  
pushMatrix();  
translate(130, 2, 0);  
rotateY(radians(90));  
ellipse(0, 0, 18, 15);  
popMatrix();
```

```
fill(180, 220, 255, 160);  
pushMatrix();  
translate(133, 2, 0);
```

```
rotateY(radians(90));  
ellipse(0, 0, 10, 8);  
popMatrix();
```

```
fill(60, 130, 200, 190);  
pushMatrix();  
translate(-48, -13, 0);  
scale(1.8, 0.55, 0.45);  
sphere(20);  
popMatrix();
```

```
fill(150, 155, 170);  
pushMatrix();  
translate(-48, -20, 0);  
box(36, 3, 3);  
popMatrix();
```

```
fill(50, 52, 62);  
pushMatrix();  
translate(-55, 8, 0);  
rotateY(radians(90));  
ellipse(0, 0, 16, 10);  
popMatrix();
```

```
fill(100, 104, 118);  
pushMatrix();  
translate(-30, 8, 0);  
box(50, 10, 14);  
popMatrix();
```

```
fill(140, 142, 158);  
pushMatrix();  
translate(18, 3, -82);  
box(30, 5, 5);  
popMatrix();
```

```
pushMatrix();  
translate(18, 3, 82);  
box(30, 5, 5);  
popMatrix();
```

```
fill(255, 50, 50);  
pushMatrix(); translate(18, 3, -88); sphere(3); popMatrix();
```

```
fill(50, 220, 80);
pushMatrix(); translate(18, 3, 88); sphere(3); popMatrix();

fill(255, 255, 255);
pushMatrix(); translate(132, 2, 0); sphere(2.5); popMatrix();
}

void serialEvent(Serial p) {
  String data = trim(p.readStringUntil('\n'));
  if (data != null && data.length() > 0) {
    String[] vals = split(data, ',');
    if (vals.length >= 3) {
      try {
        pitch = float(vals[0]);
        roll = float(vals[1]);
        yaw = float(vals[2]);
      } catch (Exception e) {
        println("Parse error: " + e.getMessage());
      }
    }
  }
}
```